講義導讀

嗨大家，這份講義內容蠻多的，這邊簡單說一下建議的閱讀順序，讓你不用從頭到尾硬啃。

第一步：基本觀念

建議先從這裡開始。這一區涵蓋了後端系統最核心的基礎知識：

- Networking Essentials
- CAP Theorem
- Scalability
- API Design
- Consistent Hashing
- Database Indexing
- Database Transactions
- Numbers to Know
- Caching
- Sharding
- Replication

這些東西就像地基，你後面看設計模式或做系統設計的時候，會一直用到這些概念。不需要一次讀完，但至少每一篇先掃過一遍，知道它在講什麼、解決什麼問題。

第二步：設計模式

有了基本觀念之後，可以來看設計模式。這邊整理了幾個實戰中很常碰到的場景：

- Scaling Reads
- Scaling Writes
- Manage Long Running Tasks
- Handling Large Blobs
- Real-time Updates
- Search System

buildmoat.org

- Data Pipeline Design
- RAG (Retrieval-Augmented Generation)

每一篇都會告訴你：遇到這類需求的時候，業界通常怎麼處理、有什麼 trade-off。讀的時候重點放在「什麼場景會用到」跟「為什麼這樣選」，比死記架構圖更重要。

時間有限的話：直接看面試模板

如果你最近就要面試、時間不夠把前面全看完，那就先跳來看面試模板。

這篇把系統設計面試拆成四個階段，每個階段該做什麼、該注意什麼都寫得很清楚：

- 系統需求（~5 min）— 釐清功能性 / 非功能性需求，挑出最重要的 3 個功能
- API 設計（~5 min）— 定義 REST / GraphQL / RPC 介面，對應到前面整理的需求
- 大方向設計（~10-15 min）— 畫架構圖、走過資料流、標出核心 entity
- 深入探討（~10 min）— 針對瓶頸做優化，展示你對 scalability / caching / sharding 的理解

看完這篇之後，如果發現自己某個概念不熟，再回去翻基本觀念或設計模式對應的章節就好。

常用技術 & 維運與可靠性：當參考資料用

剩下兩個區塊不用特別從頭讀，當工具書翻就好。

常用技術涵蓋了各種具體的技術元件：

- Database
- Blob Storage
- API Gateway
- Load Balancer
- Container
- Serverless
- Queue
- Distributed Cache
- Distributed Lock
- CDN (Content Delivery Network)

buildnoat.org

遇到系統設計題需要選技術方案的時候，回來查對應的篇章。

維運與可靠性則是進階主題：

- Dealing With Contention
- Overload Protection
- Reliable Delivery
- Observability

面試如果聊到系統穩定性或 production 經驗，這邊的內容會很有用。

總結

- 有完整時間準備 → 基本觀念 → 設計模式 → 面試模板 → 常用技術當參考
- 時間有限、快要面試 → 面試模板先看 → 不熟的概念回去補基本觀念和設計模式
- 做系統設計題卡住 → 到常用技術和維運與可靠性查對應的技術方案

祝大家準備順利！

buildmoat.org